

Assignment 2: Coding Basics

Ruixin Yang

Contents

OVERVIEW	1
Directions	1
Crafting Reports - Part 1	1
Coding Basics - Part 2	2
Coding Basics - Part 3	2
Crafting reports - Part 4	4

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Explorations on reproducibility and coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Crafting Reports - Part 1

Import packages and data, suppressing messages

Set the following R code chunk so that it runs when knit, but no messages, errors, or any output is shown. The code itself, however, should be displayed.

```
# Import libraries
library(tidyverse)
library(knitr)
```

Coding Basics - Part 2

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. Create a sequence from one to 55, increasing by fives
num_sequence <- seq(from=1, to=55, by=5)
num_sequence
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2. Compute the mean and median of this sequence
mean(num_sequence)
```

```
## [1] 26
```

```
median(num_sequence)
```

```
## [1] 26
```

```
#3. Determine whether the mean is greater than the median
mean_sequence <- mean(num_sequence)
median_sequence <- median(num_sequence)
mean_sequence > median_sequence
```

```
## [1] FALSE
```

Coding Basics - Part 3

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5. Create three vectors
#a student names (character vector)
student_names <- c("Tom", "Jerry", "Phineas", "Ferb")
#b test scores (numeric vector)
test_scores <- c(90, 80, 70, 40)
#c whether they are on scholarship or not (logical vector)
scholarship <- c(TRUE, FALSE, TRUE, FALSE)
```

```
#7. Combine each of the vectors into a data frame
```

```
student_dataset <- data.frame(student_names, test_scores, scholarship)

#8. Label the columns of the data frame with informative titles
names(student_dataset) <- c("Student_Names", "Test_Scores", "Scholarship_Status")
student_dataset
```

```
##   Student_Names Test_Scores Scholarship_Status
## 1         Tom         90             TRUE
## 2        Jerry         80             FALSE
## 3     Phineas         70             TRUE
## 4        Ferb         40             FALSE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: All elements in a matrix must have the same data type. However, different columns in a data frame can have different data types, including character values, numerical values and logical values.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
function1 <- function(score){
  if (score > 50) {
    print("Pass")
  } else {
    print("Fail")
  }
}

#11. Create a function using ifelse()
function2 <- function(score){
  print(ifelse(score>50, "Pass", "Fail"))
}

#12a. Run the first function with the value 52.5
function1(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5
function2(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
# function1(test_scores)  
  
#13b. Run the second function with the vector of test scores  
function2(test_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Fail"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: In the vector of test scores, it has 4 scores. `ifelse()` works because it can check each score in the vector separately. But `if...else` only works with one TRUE or FALSE value, so it cannot evaluate the whole vector.

Crafting reports - Part 4

Knit and submit.

Add a table of contents to your document and knit to a PDF. Submit your PDF to Canvas, but also be sure to commit and push your Rmd file used to create this knit document to GitHub. In the section below, add a link to your GitHub repository.

Github link

Answer: https://github.com/Ruixin0428/EDE_R_Lab_F26.git

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)